Operator Learning for Parametric Darcy Flow

Fourier Neural Operator (FNO) vs. MLP Baseline: Optimization, Scaling & Solver Comparison

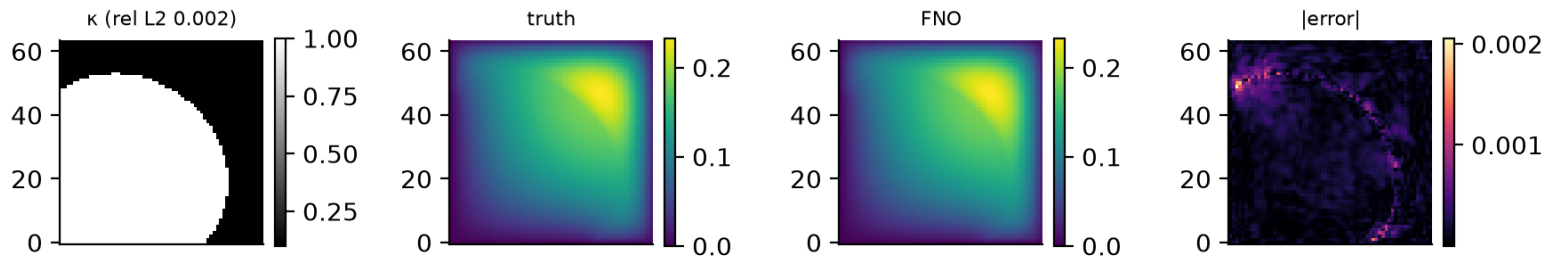
Team PINNacles

Aryamann Srivastava, Varun Sathaye, Atishay Jain, Vedant S. Tiwari

Department of Aerospace Engineering, IIT Kanpur | November 2026

The Task

Predict how fluid pressure spreads through a porous material, quickly



Left to right: permeability map (input), true pressure, FNO prediction, error

Two models learn this map from data:
FNO: sees the whole field, works with patterns in space.
MLP: a plain fully connected network, no spatial structure.

10,000
real simulations (PDEBench)

900 / 100
train / validation samples

200
held-out test samples

Error = relative difference to the true field (smaller is better)

Where We Started

Stage 2 gave a clear winner, but we suspected we could do much better

0.052
FNO error in Stage 2

0.08
MLP error in Stage 2

1.6x
FNO advantage

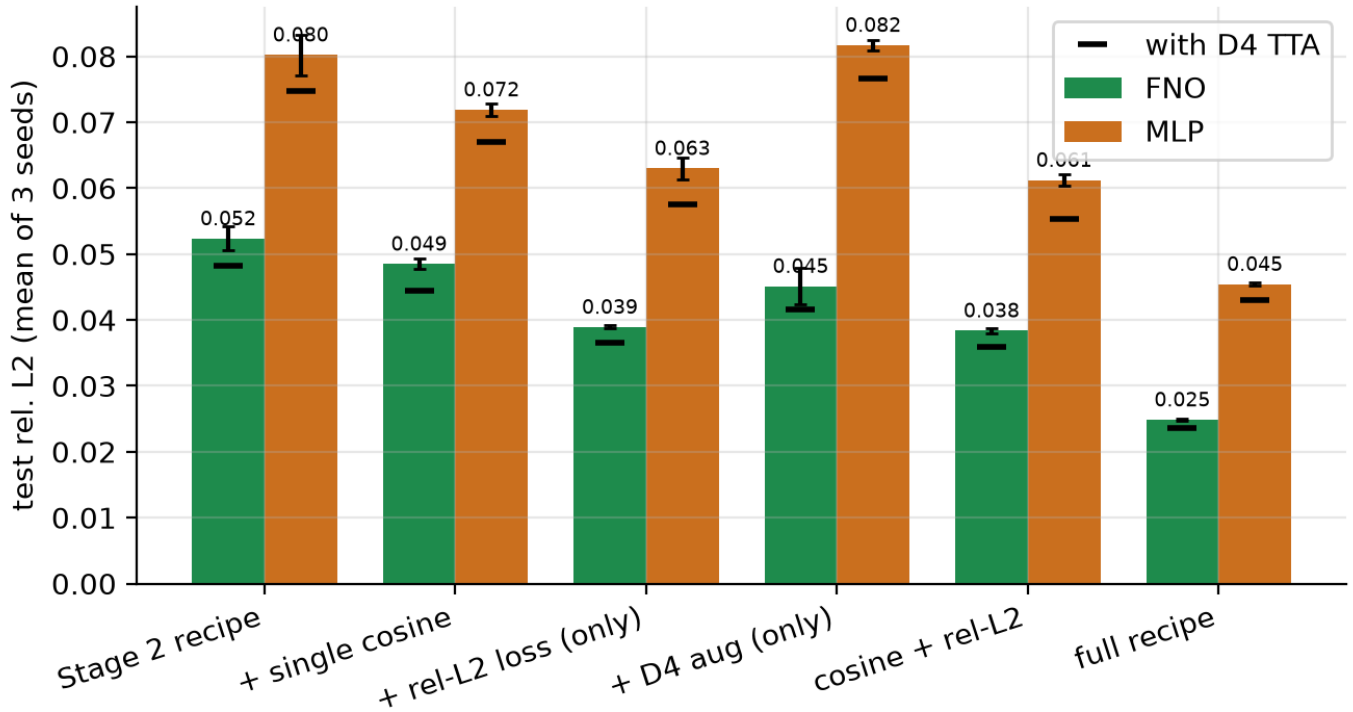
How Stage 2 trained both models

Standard squared-error loss
Learning rate that restarted again and again
No data augmentation

Question: was the remaining error about the model, or about how we trained it?

Finding 1: The Training Recipe

Changing how we train, not the model, cut the error roughly in half



1. **Match the loss to the metric**
   Train on relative error, the number we report

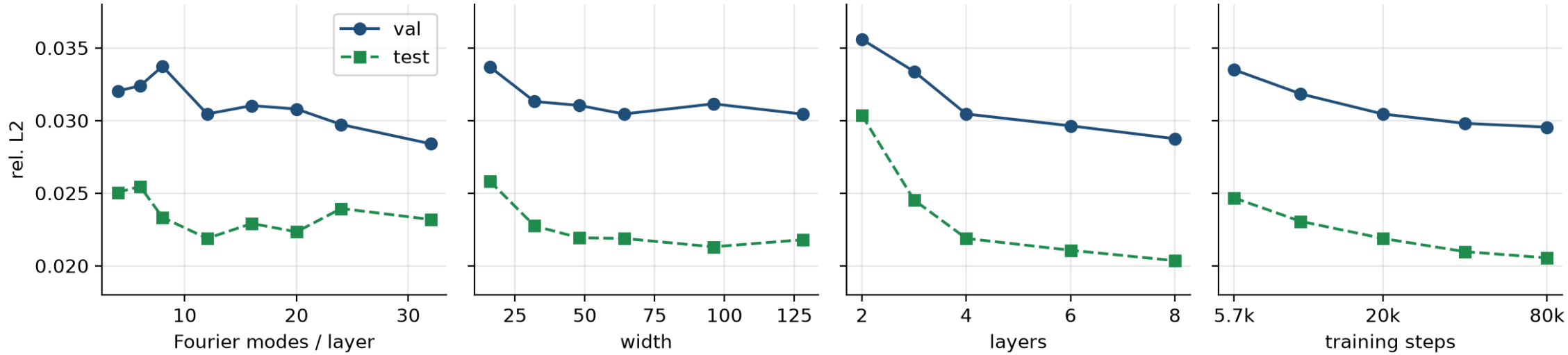
2. **One smooth learning-rate decay**
   Instead of repeated restarts

3. **Flip and rotate the inputs**
   Free extra training data (symmetries)

Same model, same budget: FNO error 0.052 to 0.025, MLP 0.080 to 0.045

Finding 2: Architecture Matters Little

Width, depth and Fourier modes were tuned one at a time on validation data



Modes, width

almost no effect past modest values

Depth, training length

small, steady gains

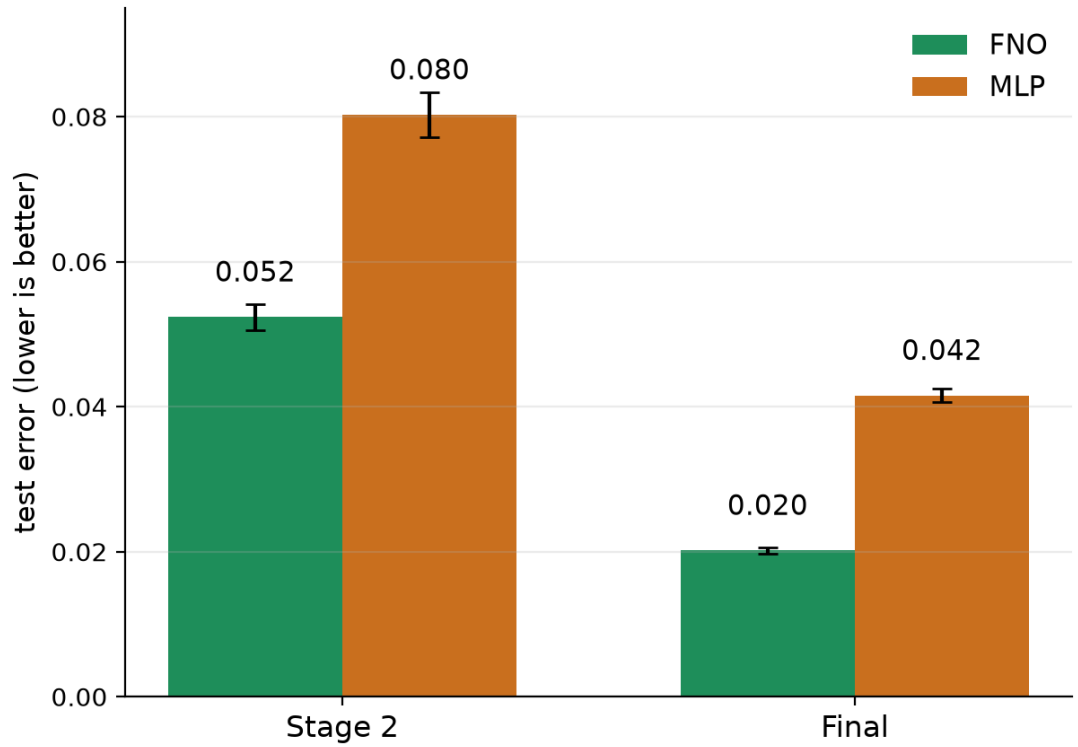
Final choice

9.5 M parameters, 8 layers

The training recipe was the big lever; architecture tuning added only a few percent

The Final Models

Both models tuned fairly, validation data only, 5 random seeds each



0.0202

FNO error, 9.5 M parameters

0.0415

MLP error, 100.7 M parameters

2x

more accurate

10x

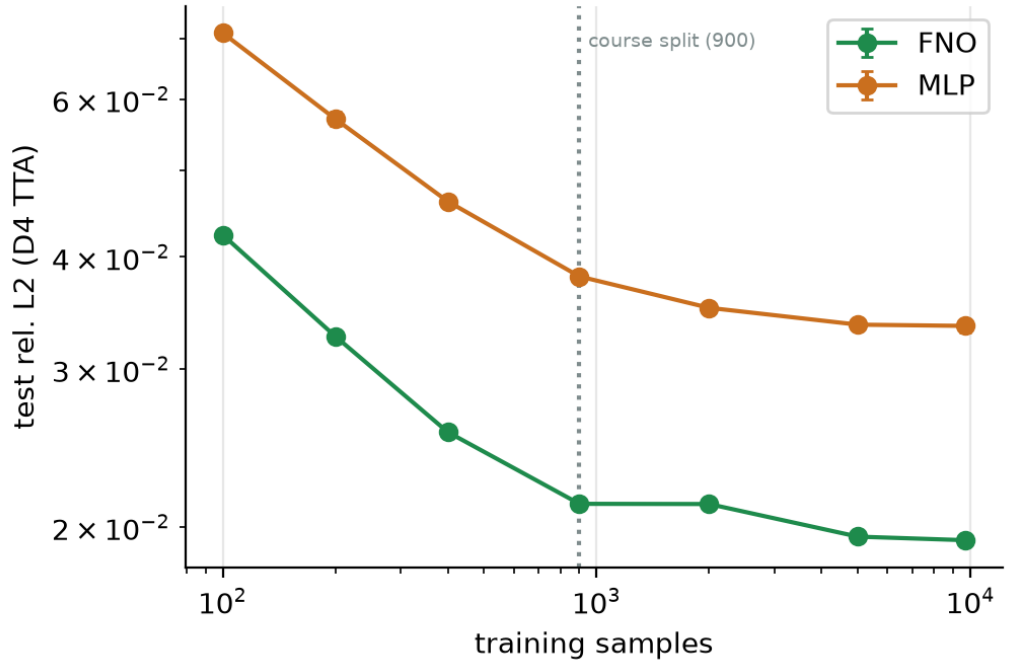
fewer parameters

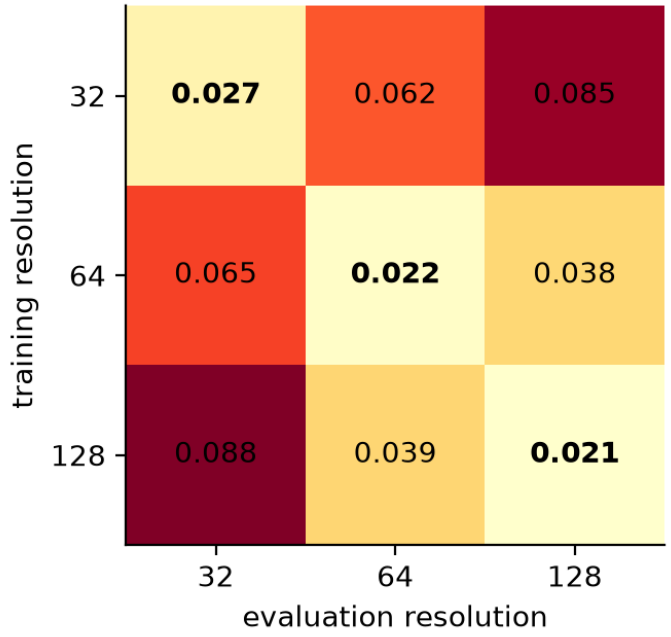
The gap between FNO and MLP grew from 1.6x to 2x once both were trained well.

Spatial structure helps: the FNO wins with a tenth of the parameters

Finding 3: Data and Resolution

The FNO needs less data and can be used on other grid sizes, with a cost





How to read the grid

Rows: grid used for training

Columns: grid used for testing

Numbers: test error

400 samples

The FNO already beats an MLP trained on 9,700

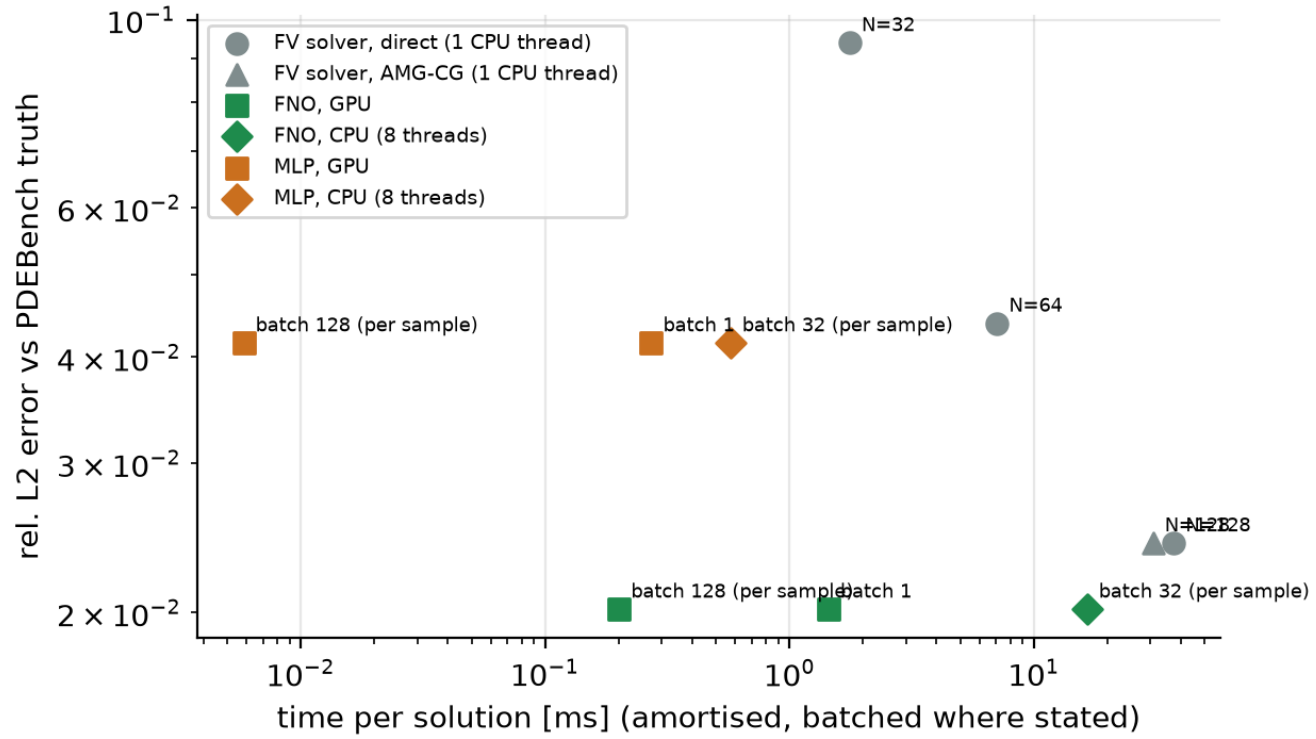
Other grid sizes

Works, but about 2 to 3x worse than a model trained on that grid

More data helps until about 5,000 samples, then error levels off near 0.02

FNO versus a Classical Solver

Compared with a finite-volume solver tuned to the same data



0.020 vs 0.044

FNO vs solver error at 64 x 64

5x to 35x

faster than the solver on a GPU

Not faster

on a single CPU core

Best when used on GPUs and in batches; the solver stays competitive on one core

Does It Obey the Physics?

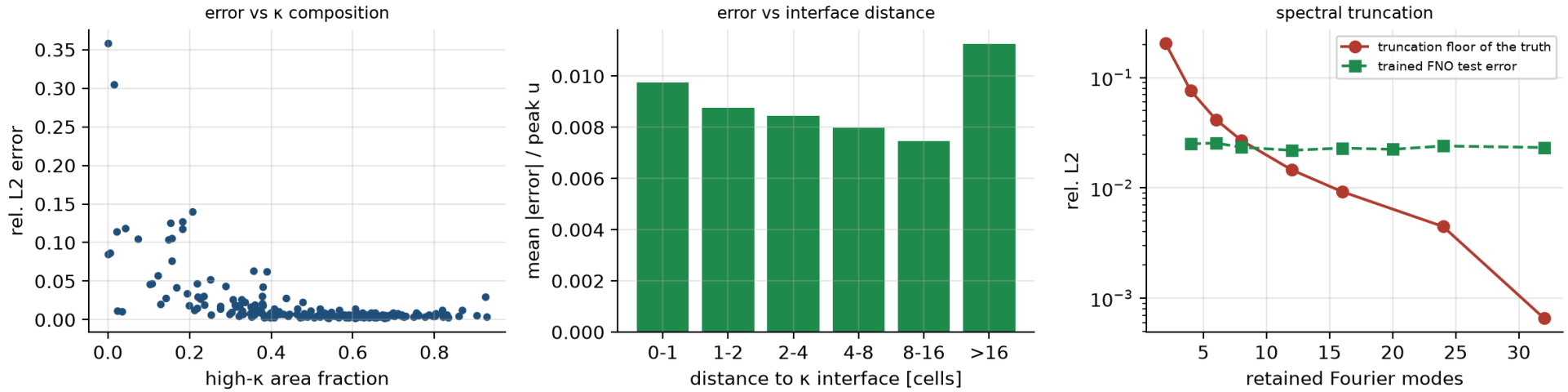
Checks beyond a single error number

0.1157
boundary level (truth 0.1159)

1.7%
error in average pressure

0.0015%
pixels with negative pressure

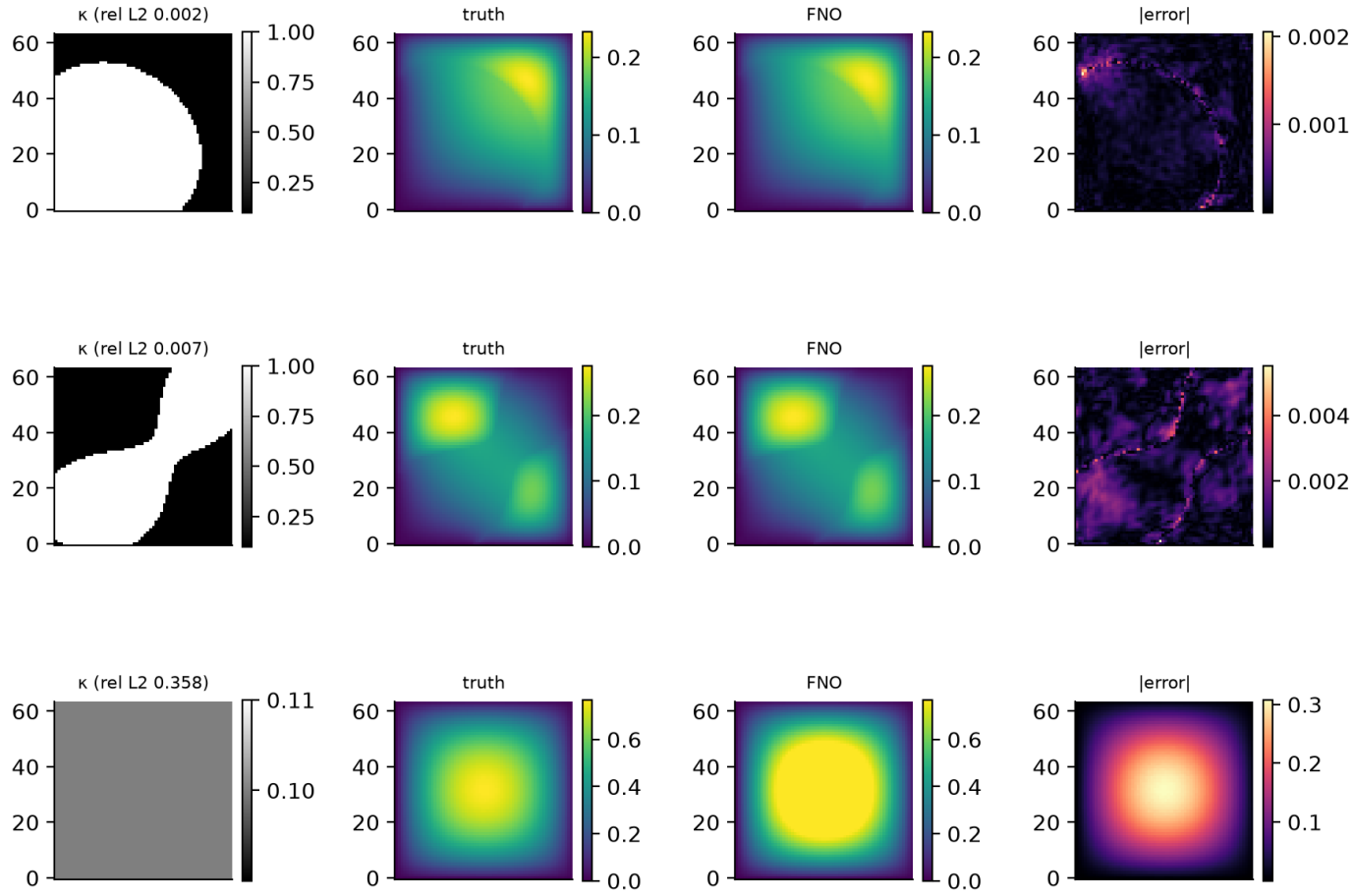
4.25 vs 4.01
equation residual (truth)



Largest errors: nearly uniform permeability with large pressure, where the amplitude is hard to infer

Example Predictions

Best, typical and worst test cases of the final FNO



1 Best

Error 0.2%, almost no visible error

2 Typical

Error 0.7%, shapes match closely

3 Worst

Uniform permeability: right shape, wrong height (36%)

Summary and Limits

What we learned and what we do not claim

What we learned

1. **Training mattered most**
   Error halved with a better recipe
2. **FNO beats a tuned MLP**
   2x more accurate, 10x fewer parameters
3. **Strong against a solver**
   More accurate at the same grid size

Honest limits

1. **One dataset**
   Results are about agreement with this data
2. **Timing caveats**
   Solver tuned to the data, shared machine
3. **Stage 2 speed claim withdrawn**
   The old reference solver was slow

Better training beat a bigger model, and the FNO stayed accurate and fast

Team Contributions & AI Declaration

Individual ownership and adherence to AE646 academic integrity guidelines

Aryamann Srivastava

Lead Developer

- Stage 3 pipeline & D4 augmentation
- Recipe & architecture studies; final models
- Finite-volume solver & latency benchmarks
- Reports, decks & code packaging

Varun Sathaye

Data & Baseline Verification

- PDEBench train/val/test split verification
- Normalisation & Stage 2 baseline checks
- Solver calibration & timing review
- Section 7 results & presentation

Atishay Jain

Architecture & Ablation Review

- Training recipe & mode/ width/depth sweeps
- Ablation tables & Pareto analysis
- Model-selection protocol verification
- Section 4-5 results & presentation

Vedant S. Tiwari

Scaling & Diagnostics

- Data-scaling & cross-resolution transfer
- Physical diagnostics & residual checks
- Failure mode analysis (uniform kappa)
- Section 6 & 8 results & presentation

AI-Tool-Use Declaration

AI tools were used for coding assistance, debugging, experiment orchestration (Colab/Kaggle GPU runtimes), and LaTeX/PPT drafting. All numerical results come from real runs on real PDEBench data; no results or metrics were generated or invented. Every line of code, equation, and conclusion is fully owned, verified, and defendable by all team members in the viva.